Họ và tên: Nguyễn Đức Bảo Trung
MSSV: 25521963
Mã lớp: IT007.R16.1

OPERATING SYSTEM

LAB 3 REPORT

SUMMARY:

Task		Tatus	Page
Section 3.5	Ex 1	Done	2
	Ex 2	Done	4
	Ex 3	Done	5
	Ex 4	Done	10

Self score: 10/10

**Note: Export file to PDF and name the file by following format:*

LAB X – <Student ID>.pdf

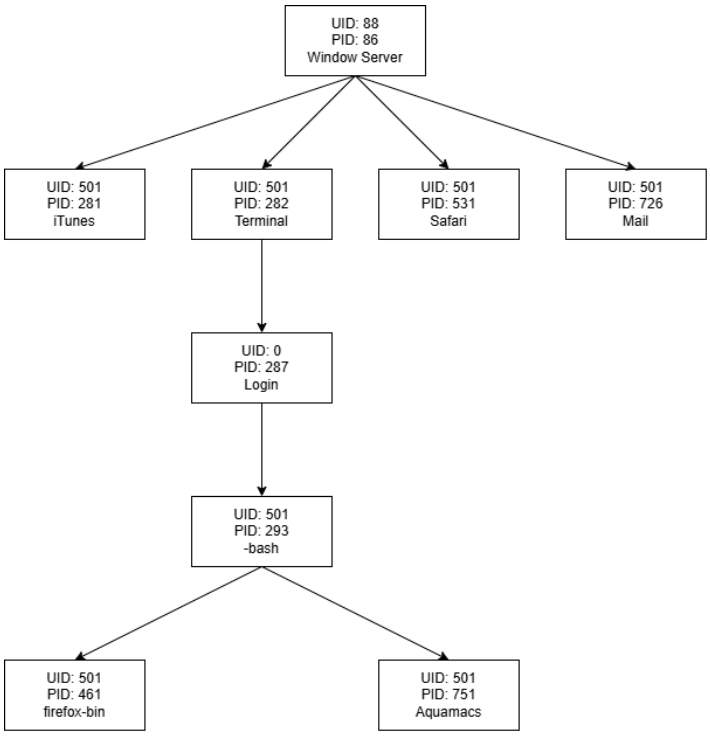
3.5 Bài tập ôn tập

1. Mối quan hệ cha-con giữa các tiến trình

a. Vẽ cây quan hệ parent-child của các tiến trình bên dưới:

UID	PID	PPID	COMMAND
88	86	1	WindowServer
501	281	86	iTunes
501	282	86	Terminal
0	287	282	login
501	461	293	firefox-bin
501	531	86	Safari
501	726	86	Mail
501	751	293	Aquamacs
501	293	287	-bash

Vẽ cây quan hệ parent – child của các tiếng trình:

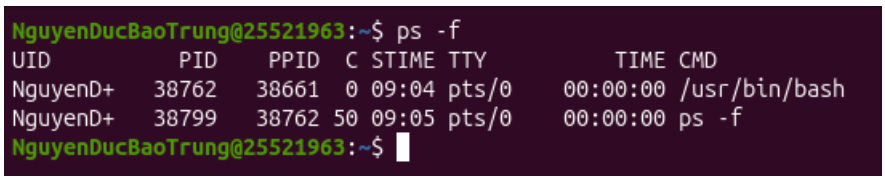


Hình 1.1 Cây tiến trình

b. Trình bày cách sử dụng lệnh ps để tìm tiến trình cha của một một tiến trình dựa vào PID của nó.

- Giống như top, ps cũng là lệnh giúp hiển thị chi tiết của tiến trình, trong đó PID là ID của tiến trình và PPID là PID của tiến trình cha của tiến trình đó.

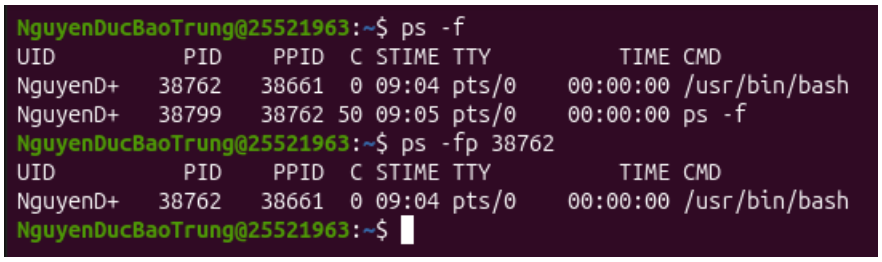
- Để sử dụng lệnh ps tìm tiến trình cha của một tiến trình dựa vào PID của nó ta dùng lệnh: **ps -f**



Hình 1.2: Kết quả khi chạy lệnh ps -f

- Trong hình ta có thể thấy rằng tiến trình bash có PID là 38762 là tiến trình cha của tiến trình ps -f nhờ lệnh ps trong qua PID và PPID của chúng.

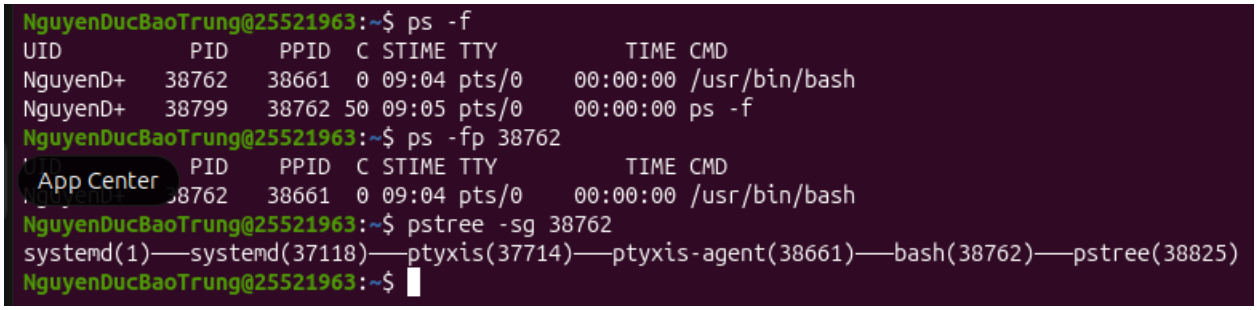
- Ta có thể sử dụng lệnh ps -fp [pidlist] để tìm PPID của tiến trình thông qua PID của tiến đó



Hình 1.3: Kết quả khi chạy lệnh ps -fp 38762

c. Tìm hiểu và cài đặt lệnh pstree (nếu chưa được cài đặt), sau đó trình bày cách sử dụng lệnh này để tìm tiến trình cha của một tiến trình dựa vào PID của nó.

- Lệnh pstree giúp hiển thị các tiến trình dưới dạng sơ đồ cây. Để sử dụng lệnh này để tìm tiến trình cha của một tiến trình dựa vào PID của nó ta dùng lệnh pstree -sg [pidlist]



Hình 1.4: Kết quả khi chạy lệnh pstree -sg 38762

- Qua kết quả trả về của lệnh pstree ta có thể thấy được tiến trình cha của bash (PID 38762) là tiến trình ptyxis-agent-(PID 38661).

2. Chương trình bên dưới in ra kết quả gì? Giải thích tại sao?

```
/*#####################################
# University of Information Technology      #
# IT007 Operating System                    #
# <Your name>, <your Student ID>            #
# File: exercise_2.c                        #
#####################################*/

#include<stdio.h>

int main(){
   pid_t pid;
   int num_coconuts = 17;
   pid = fork();
   if(pid == 0) {
      num_coconuts = 42;
      exit(0);
   } else {
      wait(NULL); /*wait until the child terminates */
   }
   printf("I see %d coconuts!\n", num_coconuts);
   exit(0);
}
```

- Chạy lại chương trình:

```
NguyenDucBaoTrung@25521963:~$ nano exercise_2.c
NguyenDucBaoTrung@25521963:~$ gcc exercise_2.c -o exercise_2
NguyenDucBaoTrung@25521963:~$ ./exercise_2
I see 17 coconuts!
NguyenDucBaoTrung@25521963:~$
```

Hình 2.1: Kết quả chạy chương trình

- Vì tiến trình con có dòng lệnh exit(0); khi tiến trình con chạy giá trị của num_coconut đã thay đổi thành 42, nhưng vì dòng lệnh trên nên dòng lệnh in kết quả ra màn hình không thực hiện được, nếu xóa dòng exit(0); này đi ta sẽ có kết quả như sau:

```
NguyenDucBaoTrung@25521963:~$ nano exercise_2.c
NguyenDucBaoTrung@25521963:~$ gcc exercise_2.c -o exercise_2
NguyenDucBaoTrung@25521963:~$ ./exercise_2
I see 42 coconuts!
I see 17 coconuts!
NguyenDucBaoTrung@25521963:~$
```

Hình 2.2: Kết quả chạy chương trình sau khi xóa dòng lệnh exit(0); trong tiến trình con.

3. Trong phần thực hành, các ví dụ chỉ sử dụng thuộc tính mặc định của pthread, hãy tìm hiểu POSIX thread và trình bày tất cả các hàm được sử dụng để làm thay đổi thuộc tính của pthread, sau đó viết các chương trình minh họa tác động của các thuộc tính này và chú thích đầy đủ cách sử dụng hàm này trong chương trình. (Gợi ý các hàm liên quan đến thuộc tính của pthread đều bắt đầu bởi: pthread_attr_*)

Hàm	Chức năng
Pthread_attr_init	Khởi tạo giá trị mặc định cho đối tượng thuộc tính
Pthread_attr_destroy	Xóa bộ nhớ được cấp phát trong quá trình khởi tạo
Pthread_attr_getschedparam	Trả về các tham số lịch trình (Scheduling Parameter) được xác định bởi pthread_attr_setschedparam ()
Pthread_attr_getschedpolicy	Để xuất scheduling policy của thread
Pthread_attr_getdetachstate	Lấy truy xuất trạng thái khởi tạo của thread, có thể thể là riêng lẻ hoặc kết hợp
Pthread_attr_getinheritsched	Trả về chính sách lịch trình (scheduling policy) được set bởi pthread_attr_setinheritsched ().
Pthread_attr_getscope	Truy xuất phạm vi của thread
Pthread_attr_setdetachstate	Sử dụng lại ID và tài nguyên của thread khi nó bị ngắt mà không phải chờ nếu thread có thuộc tính riêng lẻ.
Pthread_attr_setguardsize	Set kích thước của khu vực an toàn của thread
Pthread_attr_setstackaddr	Set địa chỉ stack của thread
Pthread_attr_setstacksize	Set kích thước stack của thread
Pthread_attr_getguardsize	Lấy kích thước của khu vực an toàn của thread
Pthread_attr_getstackaddr	Trả về địa chỉ stack của thread được set bởi pthread_attr_setstackaddr ()
Pthread_attr_getstacksize	Trả về kích thước stack của thread được set bởi pthread_attr_setstacksize ().

Pthread_attr_init

```
int pthread_attr_init(pthread_attr_t  *tattr);
#include <pthread.h>

pthread_attr_t  tattr;
int ret;

/* khởi tạo giá trị mặc định cho thuộc tính */   ret =
pthread_attr_init(&tattr);
```

Pthread_attr_destroy

```
int pthread_attr_destroy(pthread_attr_t  *tattr);
#include <pthread.h>

pthread_attr_t  tattr;  int ret;

/* Xóa thuộc tính */
ret = pthread_attr_destroy(&tattr);
```

Pthread_attr_getschedparam

```
int pthread_attr_getschedparam(pthread_attr_t  *tattr, const struct  sched_param
*param);
#include <pthread.h>

pthread_attr_t  attr;  struct sched_param  param;  int ret;

/* Lấy tham số lịch trình đang tồn tại trong thread */
ret = pthread_attr_getschedparam  (&tattr, &param);
```

Pthread_attr_getschedpolicy

```
int pthread_attr_getschedpolicy(pthread_attr_t  *tattr,  int  *policy);
#include <pthread.h>

pthread_attr_t  tattr;  int policy;  int ret;

/* Lấy chính sách lịch trình của thread */
ret = pthread_attr_getschedpolicy  (&tattr, &policy);
```

Pthread_attr_getdetachstate

```
int pthread_attr_getdetachstate(const  pthread_attr_t  *tattr,
int *detachstate;
#include <pthread.h>

pthread_attr_t  tattr;  int detachstate;  int ret;

/* Lấy trạng thái khởi tạo của thread */
ret = pthread_attr_getdetachstate  (&tattr,  &detachstate);
```

Pthread_attr_getinheritsched

```
int pthread_attr_getinheritsched(pthread_attr_t  *tattr,  int *inherit);
#include <pthread.h>

pthread_attr_t  tattr;  int inherit;  int ret;

/* Lấy chính sách lịch trình và độ ưu tiên của thread được khởi tạo */  ret =
pthread_attr_getinheritsched  (&tattr, &inherit);
```

Pthread_attr_getscope

```
int pthread_attr_getscope(pthread_attr_t  *tattr,  int  *scope);
#include <pthread.h>

pthread_attr_t  tattr;  int scope;  int ret;

/* Lấy phạm vi của thread */  ret = pthread_attr_getscope(&tattr,  &scope);
```

Pthread_attr_setdetachstate

```
int pthread_attr_setdetachstate(pthread_attr_t  *tattr,int  detachstate);
#include <pthread.h>

pthread_attr_t  tattr;  int ret;

/* Set trạng thái riêng lẻ cho thread */
ret=pthread_attr_setdetachstate(&tattr,PTHREAD_CREATE_DETACHED);
```

Pthread_attr_setguardsize

```
#include <pthread.h>
int pthread_attr_setguardsize(pthread_attr_t  *attr,  size_t  guardsize);
```

Pthread_attr_setstackaddr

```
int pthread_attr_setstackaddr(pthread_attr_t  *tattr,void  *stackaddr);
#include <pthread.h>
pthread_attr_t  tattr;
void *base; int ret;

base = (void *) malloc(PTHREAD_STACK_MIN  + 0x4000);

/* Đặt địa chỉ mới cho thread */
ret = pthread_attr_setstackaddr(&tattr,  base);
```

Pthread_attr_setstacksize

```
int pthread_attr_setstacksize(pthread_attr_t  *tattr,  int size);
#include <pthread.h>

pthread_attr_t tattr;
int size;
int ret;

size = (PTHREAD_STACK_MIN  + 0x4000);
```

```
/* Đặt kích thước mới */
ret = pthread_attr_setstacksize(&tattr,  size);
```

Pthread_attr_getguardsize

```
#include <pthread.h>
int pthread_attr_getguardsize(const  pthread_attr_t  *attr, size_t  *guardsize);
```

Pthread_attr_getstackaddr

```
int pthread_attr_getstackaddr(pthread_attr_t  *tattr,void  * *stackaddr);
#include <pthread.h>

pthread_attr_t  tattr;
void *base;  int ret;

/* Lấy một địa chỉ mới */
ret = pthread_attr_getstackaddr  (&tattr, *base);
```

Pthread_attr_getstacksize

```
int pthread_attr_getstacksize(pthread_attr_t  *tattr,  size_t *size);
#include <pthread.h>

pthread_attr_t  tattr;
int size;  int ret;

/* Lấy kích thước stack */
ret = pthread_attr_getstacksize(&tattr,  &size);
```

4. Viết chương trình làm các công việc sau theo thứ tự:

a. In ra dòng chữ: “Welcome to IT007, I am <your_Student_ID>!”

```
NguyenDucBaoTrung@25521963:~$ nano exercise_4.c
NguyenDucBaoTrung@25521963:~$ gcc exercise_4.c -o exercise_4
NguyenDucBaoTrung@25521963:~$ ./exercise_4
Welcome to IT007, I am 25521963!
NguyenDucBaoTrung@25521963:~$
```

Hình 4.1: Kết quả câu a

b. Mở tệp abcd.txt bằng vim editor

```
NguyenDucBaoTrung@25521963:~$ vim abcd.txt
NguyenDucBaoTrung@25521963:~$
```

Hình 4.2: Mở file abcd.txt bằng vim



Hình 4.3: Nội dung file abcd.txt khi mở bằng vim

c. Tắt vim editor khi người dùng nhấn CRTL+C

- dùng Key Mapping cấu hình ngay trong file của Vim để làm điều này, khi mở 1 file vim thì ấn Ctrl + C nó sẽ thoát thay vì Esc + : w q

```
 GNU nano 8.7.1    /home/NguyenDucBaoTrung/.vimrc
nnoremap <C-c> :q!<CR>

inoremap <C-c> <Esc>:q!<CR>

vnoremap <C-c> <Esc>:q!<CR>
```

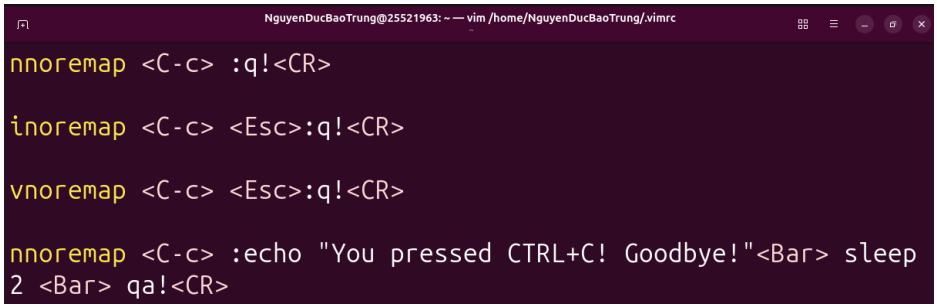
Hình 4.4: Nội dung file .vimrc

- Lệnh đầu tiên: nnoremap <C-c> :q!<CR>
Giải thích: Khi ở Normal mode, nhấn Ctrl+C sẽ thoát thẳng (bỏ qua lưu)

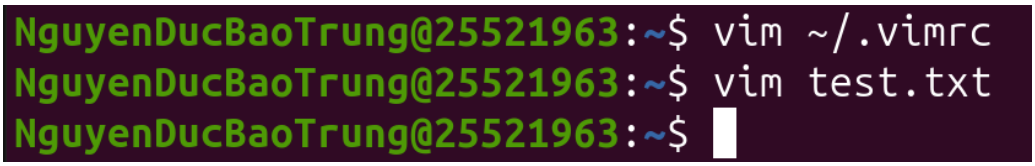
- Lệnh thứ hai: inoremap <C-c> <Esc>:q!<CR>
Giải thích: Khi ở Insert mode (đang gõ văn bản), nhấn Ctrl+C cũng thoát thẳng

- Lệnh thứ ba: vnoremap <C-c> <Esc>:q!<CR>
Giải thích: Khi ở Visual mode (đang bôi đen), nhấn Ctrl+C cũng thoát thẳng

d. Khi người dùng nhấn CTRL+C thì in ra dòng chữ: “You are pressed CTRL+C! Goodbye!”



Hình 4.5: Nội dung file .vimrc



Hình 4.6: Mở file test.txt



Hình 4.7: Sau khi ấn Ctrl + C thì hiển thị dòng đã bấm Ctrl + C